

GraphCodeRAG

Graph-Enhanced Code Retrieval-Augmented Generation Project Documentation

AI for Engineers — Spring 2026

Group 4

Lokeshwar Reddy Peddarangareddy

Nagraj Venkatesham Vallakati

Neha Reddy Poreddy

Rahul Sarma Vogeti

Swet Vimalkumar Patel

1. Keywords and Their Meanings

Before diving into the project, here are the key technical terms used throughout this document. Each is explained in plain language so that anyone — regardless of technical background — can follow the rest of the documentation.

Term	Meaning
RAG (Retrieval-Augmented Generation)	A technique where an AI first retrieves relevant documents from a database, then uses them as context to generate an answer. Instead of relying on memory alone, the AI reads real data before answering.
AST (Abstract Syntax Tree)	A tree-shaped representation of code structure. Each node represents a code element: a function, a class, an if-statement, a variable assignment. It lets us understand code structure programmatically rather than treating it as plain text.
Embedding	A mathematical representation of text as a vector (a list of numbers). Similar texts produce similar vectors. A 768-dimensional embedding means each piece of code is represented as a list of 768 numbers. The model we use (CodeRankEmbed) is specifically trained on code.
FAISS	Facebook AI Similarity Search. An open-source library for fast nearest-neighbor search in high-dimensional vector spaces. It lets us quickly find which code chunks are most similar to a query by comparing their embedding vectors.
Cosine Similarity	A measure of how similar two vectors are, based on the angle between them. A cosine similarity of 1.0 means the vectors point in the same direction (identical meaning). A value of 0.0 means they are unrelated.
Knowledge Graph	A database that stores entities (things) and relationships (connections between things) as a network of nodes and edges. In our case, nodes are functions/classes/modules and edges are IMPORTS/CALLS/CONTAINS/INHERITS.
Neo4j	A graph database that stores data as nodes and relationships rather than tables. We query it using Cypher, a language similar to SQL but designed for graph traversal.
Tree-sitter	An open-source parsing library that can read source code in many languages and build an AST. Unlike Python's built-in ast module, Tree-sitter handles broken or incomplete code without crashing.
Hybrid Retrieval	Our approach of combining vector search (finding similar text) with graph traversal (finding connected code) and merging the results. This is the core innovation of the project.
Two-Tier Chunking	Our strategy of keeping small code chunks whole while splitting large ones into overlapping children. The children are embedded for precise search; the parent is returned for complete context.
Score Fusion	The process of combining scores from different sources (vector similarity and graph proximity) into a single ranking. We use a weighted formula: 75% vector score + 25% graph proximity.

MRR (Mean Reciprocal Rank)	A metric that measures how high the first correct result appears. If the correct file is ranked 1st, MRR = 1.0. If 3rd, MRR = 0.33. Averaged across all queries.
NDCG (Normalized Discounted Cumulative Gain)	A metric that rewards systems for placing multiple relevant results high in the ranking, not just the first one. It penalizes relevant results that appear lower in the list.
Recall@K	The fraction of all correct files that appear in the top K results. If there are 3 correct files and you find 2 of them in the top 10, Recall@10 = 66.7%.
Hit Rate@K	A binary metric: 1 (100%) if at least one correct file appears in the top K results, 0 (0%) otherwise. The simplest pass/fail measure.
SWE-bench	A benchmark dataset of real-world bug reports from open-source Python projects (Click, PyTest, Django, Scikit-Learn). Each bug report comes with the files that were actually modified in the fix, providing ground truth for evaluation.
LLM-as-Judge	Using one AI model (Gemini 2.0 Flash) to evaluate the quality of answers produced by another AI model. This avoids the bias that would occur if a model judged its own outputs.
CodeRankEmbed	The embedding model we use (nomic-ai/CodeRankEmbed). It has 137 million parameters and produces 768-dimensional vectors. It is specifically trained on code, so it understands programming concepts better than general-purpose text models.
Ollama	A tool for running large language models locally on your own computer. We use it to run Qwen 2.5 Coder 7B without needing any API keys or cloud costs.
FastAPI	A modern Python web framework for building REST APIs. We use it to serve the web interface with security features including CORS, SSRF prevention, and XSS mitigation.

2. What Is GraphCodeRAG?

Imagine you are a developer working on a large codebase with thousands of files. You have a question like "How does the login function validate passwords?" You could search through the code manually, but that takes a long time. What if an AI could read your entire codebase and answer questions about it?

That is exactly what GraphCodeRAG does. It is an AI-powered question-answering system built specifically for code. You feed it a Python repository, and it lets you ask natural language questions about how the code works. It then finds the most relevant pieces of code and generates a clear, cited answer.

2.1 What Makes It Different?

Most existing code search tools use a technique called RAG (Retrieval-Augmented Generation). Standard RAG works like this: it breaks your code into small chunks of text, converts each chunk into a mathematical representation (called an embedding), stores these embeddings in a database, and when you ask a question, it finds the chunks whose embeddings are most similar to your question. This works reasonably well, but it has a major blind spot.

Standard RAG only finds code that looks like your question. It misses code that is structurally connected to your question. For example, if you ask about the "Command" class, standard RAG might find the class definition but miss the "Group" class that inherits from it, or the "decorators.py" file that creates Command objects. These connections are invisible to text similarity — but they are exactly what a developer needs to understand the code.

GraphCodeRAG solves this by combining two search strategies. First, it does the standard text similarity search (vector search). Second, it builds a knowledge graph — a map of how code entities (functions, classes, modules) are connected through imports, function calls, inheritance, and containment. When you ask a question, it searches both the text index and the graph, then merges the results intelligently. This hybrid approach finds code that is both semantically relevant and structurally connected.

3. Architecture: Step-by-Step Walkthrough

The system has five main stages. Think of it like a library: first you catalog all the books (ingestion), then you organize them on shelves and build a card catalog (storage), then when someone asks a question you find the right books (retrieval), and finally you read the relevant pages and give an answer (generation). Here is each stage in detail.

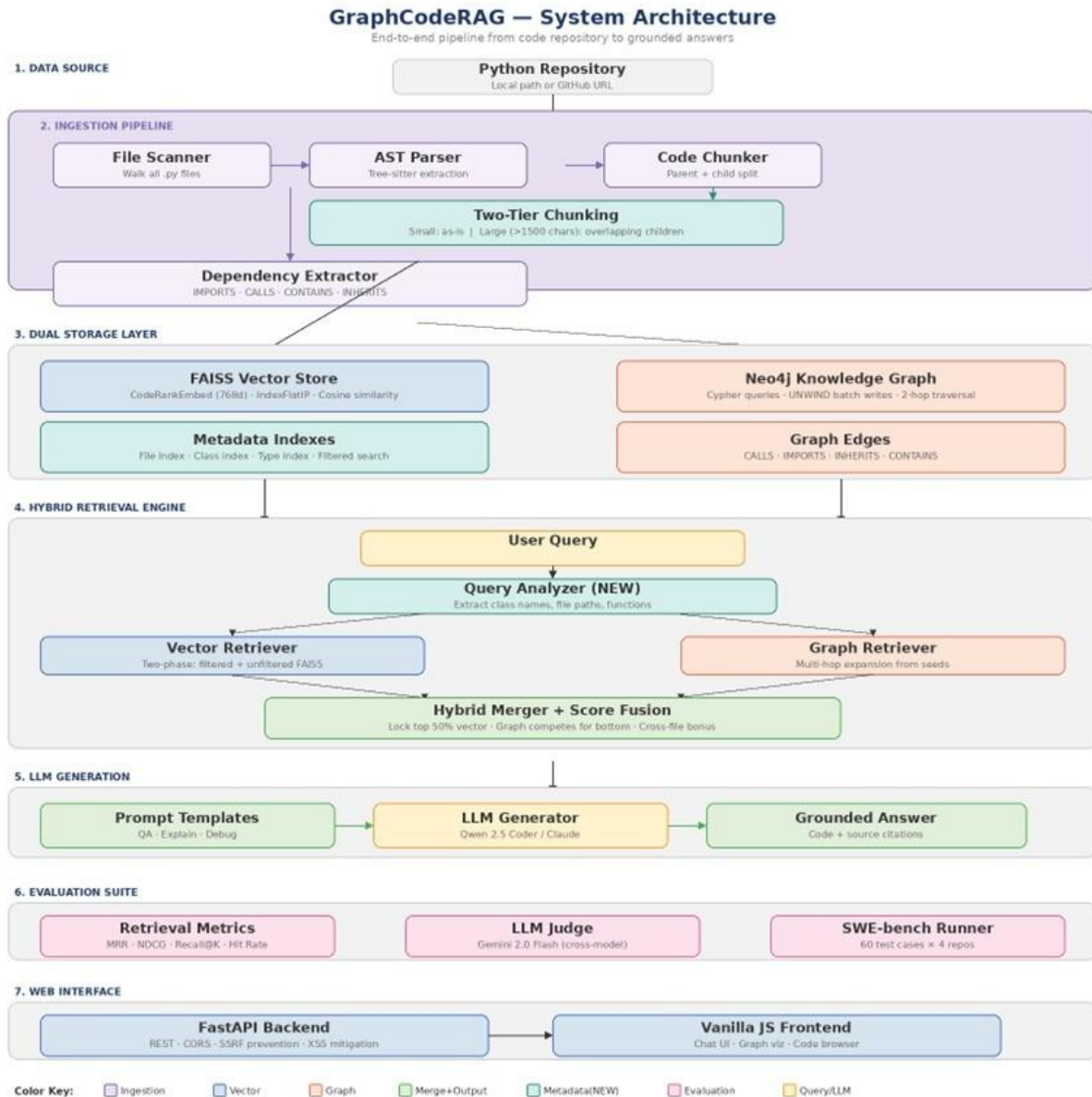


Figure 1: GraphCodeRAG System Architecture

3.1 Step 1 — File Scanning

What it does: The system walks through your Python repository and collects all .py files. It automatically skips folders that are not part of the actual codebase — things like `__pycache__`, `.git`, `venv`, and `node_modules`. Think of this as walking through a library and pulling only the actual books off the shelves, while ignoring the cleaning supplies and furniture.

How we did it: We wrote a `file_scanner.py` module that uses Python's `os.walk()` to recursively traverse the directory tree. It has a configurable exclusion list so you can add or remove folders to skip. The output is a simple list of file paths.

3.2 Step 2 — AST Parsing (Understanding Code Structure)

What it does: For each Python file, the system reads the code and understands its structure — which functions exist, which classes exist, which methods belong to which class, what the function signatures look like, and what the docstrings say. This is like reading a book's table of contents before reading the actual pages.

How we did it: We used Tree-sitter, an open-source parser that builds an Abstract Syntax Tree (AST) from source code. We chose Tree-sitter over Python's built-in `ast` module because Tree-sitter can handle broken or incomplete code without crashing — which is common in real-world repositories where files might have syntax errors. The parser extracts every function, class, and method as an `ASTNode` object with metadata: name, source code, line numbers, docstring, signature, decorators, and parent class.

3.3 Step 3 — Code Chunking (Breaking Code into Searchable Pieces)

What it does: The system converts each AST node into a "chunk" a self-contained piece of code that can be searched independently. Each function becomes one chunk. Each class (with all its methods) becomes one chunk. Module-level code (imports, constants) becomes a separate chunk.

The two-tier strategy: Here is the clever part. Short chunks (under 1,500 characters) are stored as-is. But long chunks (a 500-line function, for example) create a problem: when you convert a very long piece of text into a single embedding vector, the meaning gets diluted — the embedding becomes a vague average of everything in the function rather than a precise representation of any specific part.

To solve this, we split long chunks into overlapping "child" snippets (1,500-character windows with 200-character overlap). Each child gets a context header injected at the top ("`# File: core.py`, `# Class: Command`, `# Function: parse_args`") so the embedding captures not just the code but also where it lives. The full parent chunk is stored in metadata but not embedded. Only the children go into the vector index. When a child matches a query, the system looks up its parent and returns the complete function — giving the AI the full context rather than a fragment.

3.4 Step 4 — Dependency Extraction (Building the Graph)

What it does: The system walks the AST a second time, this time looking for relationships between code entities. It extracts four types of directed edges:

Edge Type	What It Captures
IMPORTS	File A imports something from File B. Example: <code>decorators.py</code> imports <code>Command</code> from <code>core.py</code> .
CALLS	Function A calls Function B inside its body. Example: <code>invoke()</code> calls <code>parse_args()</code> .
CONTAINS	A class contains a method. Example: the <code>Command</code> class contains the <code>parse_args</code> method.
INHERITS	Class A extends Class B. Example: <code>Group</code> inherits from <code>Command</code> .

How we did it: The `dependency_extractor.py` module uses regex and AST walking to find import statements, function calls within function bodies, class-method containment, and base class declarations. These edges are stored as a list of (source, target, edge_type) triples that get loaded into Neo4j.

3.5 Step 5 — Dual Storage (Vector Index + Knowledge Graph)

All the chunks and edges are stored in two separate databases that serve different purposes:

FAISS Vector Store: Each chunk is converted into a 768-dimensional embedding vector using the CodeRankEmbed model. These vectors are stored in a FAISS index (IndexFlatIP) that supports fast inner-product search, which is equivalent to cosine similarity when vectors are normalized. Think of this as a "meaning-based" index — chunks with similar code or descriptions end up close together in this 768-dimensional space.

Neo4j Knowledge Graph: The IMPORTS, CALLS, CONTAINS, and INHERITS edges are stored in a Neo4j graph database. Each code entity (function, class, module) becomes a labeled node, and each relationship becomes a directed edge. This lets us run queries like "find everything that calls `parse_args`" or "find all classes that inherit from `Command`" using Neo4j's Cypher query language.

3.6 Step 6 — Hybrid Retrieval (Finding the Right Code)

This is the core innovation. When a user asks a question, the system runs three sub-steps:

Step 6a — Vector Search: The query is embedded into the same 768-dimensional space as the code chunks, and FAISS finds the top-K chunks with the highest cosine similarity. This finds code that "sounds like" the question.

Step 6b — Graph Expansion: The system takes the best vector results (one per unique file, to avoid clustering) and uses them as "seeds" in the knowledge graph. From each seed, it traverses up to 2 hops following CALLS, IMPORTS, and INHERITS edges. This discovers structurally related code that vector search might have missed — the callers, callees, parent classes, and imported modules.

Step 6c — Hybrid Merge: The results from both sources are combined using a score fusion algorithm. The top 50% of vector results are "locked" (never displaced). Graph-discovered chunks are scored using the formula: $\text{hybrid_score} = 0.75 \times \text{cosine_similarity} + 0.25 \times \text{graph_proximity}$, with a 15% bonus for cross-file results. These graph chunks compete with the bottom 50% of vector results for the remaining slots. The final ranked list contains the best of both worlds.

3.7 Step 7 — LLM Generation (Producing the Answer)

What it does: The merged, ranked code chunks are assembled into a prompt alongside the user's question. This prompt is sent to an LLM (either Qwen 2.5 Coder 7B running locally via Ollama, or Anthropic Claude via API) which reads the code context and produces a grounded answer with source citations. Three prompt templates are available: QA (factual), Explain (detailed walkthrough), and Debug (root cause analysis with fix suggestions).

4. How Relevance Relates to Ranking

4.1 What Is Relevance?

A code chunk is "relevant" if it belongs to a file that is part of the correct answer. In our evaluation, we use SWE-bench bug reports where the ground truth is the set of files that were actually modified to fix the bug. If a bug fix touched `core.py` and `decorators.py`, then any chunk from those two files is considered relevant.

4.2 How Does the System Rank Results?

The system produces a ranked list of code chunks, ordered from most relevant to least relevant. The ranking comes from three scoring mechanisms that work together:

Scoring Mechanism 1 — Cosine Similarity (Vector Score): When you type a question, the system converts it into a 768-dimensional embedding vector. It then computes the cosine similarity between your question's vector and every code chunk's vector. The higher the similarity, the higher the rank. For example, if you ask, "How does Click parse arguments?", a chunk containing the `parse_args()` function will have a high cosine similarity because the words and concepts overlap.

Scoring Mechanism 2 — Graph Proximity (Structural Score): For chunks discovered via graph traversal, the system computes a proximity score based on how many hops away the chunk is from the vector search seeds. A chunk that is directly called by a seed function (1 hop) gets a higher proximity score than a chunk that is two hops away. The formula is: $\text{graph_proximity} = 1 / (1 + \text{distance})$. So 1 hop gives a proximity of 0.50, and 2 hops gives 0.33.

Scoring Mechanism 3 — Hybrid Score Fusion: For graph-discovered chunks, the final score is a weighted combination: $\text{hybrid_score} = 0.75 \times \text{cosine_similarity} + 0.25 \times \text{graph_proximity}$. If the chunk comes from a different file than the vector results (cross-file), it gets a 15% bonus (multiply by 1.15). This bonus rewards the graph for finding structurally connected code in files that vector search alone would not have surfaced.

4.3 The Lock-and-Compete Merge

The final ranking uses a lock-and-compete strategy:

1. The top 50% of vector results are LOCKED. They are guaranteed to appear in the final output because they have the highest semantic relevance. If $K=10$, the top 5 vector results are locked.
2. The bottom 50% of vector results enter a COMPETITION with graph-discovered chunks. Each graph chunk's hybrid score is compared against the bottom vector scores. If a graph chunk scores higher, it displaces the weaker vector result.
3. Up to 3 bonus slots are added for graph chunks that score above a minimum threshold (0.3) but did not displace any vector results. This allows the graph to contribute even when vector results are strong.
4. The final list is sorted by score: locked results first (in their original order), followed by the competition winners sorted by their hybrid scores.

4.4 Worked Example

Suppose you ask: "How does the Group class handle subcommands?"

Vector search returns 10 chunks, all from `core.py`. The top 5 (locked) include `Group.add_command` and `Group.__init__`. The bottom 5 are weaker matches like `Command.main` and `Command.make_context`.

Graph expansion starts from the Group node and traverses: Group --INHERITS--> Command (1 hop), Group --CALLS--> MultiCommand.list_commands (1 hop), `decorators.py` --IMPORTS--> Group (1 hop, cross-file). The `decorators.py` chunk has `cosine_similarity` = 0.60 and `graph_proximity` = 0.50, giving `hybrid_score` = $0.75 \times 0.60 + 0.25 \times 0.50 = 0.575$. With the 15% cross-file bonus: $0.575 \times 1.15 = 0.661$. If this beats the weakest bottom-5 vector result, it takes its slot.

The final result now contains both the Group class definition (from vector search) and the decorator wrappers (from graph traversal) giving the AI a complete picture of how subcommands work across files.

5. Evaluation Metrics Explained

We use five metrics to measure how well the system retrieves relevant code. Each metric answers a different question. Below, every metric is explained with a concrete example using the same scenario.

Running example: Suppose a bug fix requires changes to 3 files: `core.py`, `decorators.py`, and `utils.py`. The system returns a ranked list of 10 results. In this example, the system returned: `core.py` (rank 1), `test_basic.py` (rank 2), `core.py` (rank 3), `decorators.py` (rank 4), `config.py` (rank 5), `helpers.py` (rank 6), `types.py` (rank 7), `utils.py` (rank 8), `testing.py` (rank 9), `models.py` (rank 10).

5.1 MRR (Mean Reciprocal Rank)

Question it answers: "How high is the FIRST correct result?"

MRR looks at where the first relevant file appears in the ranked list. The score is 1 divided by that position. If the first correct file is at rank 1, $MRR = 1/1 = 1.0$. If it is at rank 3, $MRR = 1/3 = 0.33$. If it is at rank 10, $MRR = 1/10 = 0.10$. If no correct file appears at all, $MRR = 0$.

In our example: The first correct file is `core.py` at rank 1. So $MRR = 1/1 = 1.0$ (perfect score). If `core.py` had been at rank 4 instead, MRR would be $1/4 = 0.25$.

Why it matters: A high MRR means developers do not need to scroll through results — the right code is right at the top.

Scenario	First Correct Position	MRR Score
Best case: correct file at rank 1	1	$1/1 = 1.000$
Correct file at rank 3	3	$1/3 = 0.333$
Correct file at rank 10	10	$1/10 = 0.100$
No correct file found	—	0.000

5.2 File Recall @ K

Question it answers: "What fraction of the correct files did we find?"

File Recall counts how many of the ground-truth files appear anywhere in the top K results, divided by the total number of ground-truth files. It does not care about order — only about completeness.

In our example (K=5): The top 5 results contain core.py and decorators.py (2 out of 3 correct files). $\text{Recall}@5 = 2/3 = 66.7\%$. Note that utils.py is at rank 8, outside the top 5, so it is missed.

In our example (K=10): All 10 results are considered. core.py, decorators.py, and utils.py are all present. $\text{Recall}@10 = 3/3 = 100\%$.

K Value	Correct Files Found	Recall@K
K=1	1 of 3 (core.py)	$1/3 = 33.3\%$
K=5	2 of 3 (core.py, decorators.py)	$2/3 = 66.7\%$
K=10	3 of 3 (all found)	$3/3 = 100.0\%$

5.3 NDCG (Normalized Discounted Cumulative Gain)

Question it answers: "Are the correct results ranked HIGH, not just present?"

NDCG goes further than Recall. Recall only cares whether correct files are found — it treats rank 1 and rank 10 equally. NDCG rewards systems that place correct files higher in the list. It works by assigning a "gain" for each correct result, then "discounting" that gain based on position. A correct file at rank 1 contributes more to the score than one at rank 8.

The formula is: for each correct result at position i , the gain is $1/\log_2(i+1)$. Sum these gains (this is the DCG). Then divide by the ideal DCG (what you would get if all correct files were at the very top). The result is between 0 and 1.

In our example (K=10): Correct files are at ranks 1, 4, and 8. $\text{DCG} = 1/\log_2(2) + 1/\log_2(5) + 1/\log_2(9) = 1.000 + 0.431 + 0.316 = 1.747$. The ideal DCG (correct at ranks 1, 2, 3) = $1.000 + 0.631 + 0.500 = 2.131$. $\text{NDCG} = 1.747/2.131 = 0.820$.

Why it matters: A system with $\text{Recall}@10 = 100\%$ but all correct files at ranks 8, 9, 10 would have a low NDCG. A system with the same recall but correct files at ranks 1, 2, 3 would have $\text{NDCG} = 1.0$. NDCG captures ranking quality, not just retrieval success.

5.4 Hit Rate @ K

Question it answers: "Did we find AT LEAST ONE correct file in the top K?"

This is the simplest metric — a binary yes/no. If at least one correct file appears in the top K results, $\text{Hit Rate} = 1$ (100%). Otherwise, $\text{Hit Rate} = 0$ (0%).

In our example: core.py is at rank 1, so $\text{Hit Rate}@1 = 100\%$. Even $\text{Hit Rate}@1$ is perfect here. But for harder queries where the first correct file appears at rank 6, $\text{Hit Rate}@5$ would be 0% while $\text{Hit Rate}@10$ would be 100%.

5.5 Our Results

Here are the actual results from our evaluation, comparing Standard RAG against GraphCodeRAG across 4 repositories at K=5:

Repo (size)	Metric	Standard RAG	GraphCodeRAG	Improvement
Click (20K LOC)	MRR@5	81.7%	93.3%	+11.6
Click (20K LOC)	File Recall@5	66.7%	72.2%	+5.6
Click (20K LOC)	NDCG@5	63.6%	74.6%	+11.0
PyTest (50K LOC)	MRR@5	26.7%	41.1%	+14.4
PyTest (50K LOC)	File Recall@5	26.7%	53.3%	+26.7
PyTest (50K LOC)	NDCG@5	26.7%	44.2%	+17.5
Django (300K LOC)	MRR@5	22.2%	60.0%	+37.8
Django (300K LOC)	File Recall@5	26.7%	66.7%	+40.0
Django (300K LOC)	NDCG@5	23.3%	61.8%	+38.4
Scikit-Learn (200K LOC)	MRR@5	13.3%	23.3%	+10.0
Scikit-Learn (200K LOC)	File Recall@5	20.0%	26.7%	+6.7
Scikit-Learn (200K LOC)	NDCG@5	15.1%	24.2%	+9.1

6. Suggestions for Future Work

6.1 Multi-Language Support

The system currently only works with Python repositories. Tree-sitter supports over 40 programming languages (JavaScript, TypeScript, Java, Go, Rust, C++, and more). Extending the AST parser and dependency extractor to handle additional languages would make the system useful for polyglot codebases. The biggest challenge is not parsing (Tree-sitter handles that) but dependency extraction — each language has different import/module systems.

6.2 Incremental Ingestion

Currently, the system re-ingests the entire repository from scratch every time. For large codebases like Django (300K+ lines of code), this takes several minutes. Implementing incremental ingestion — detecting which files changed since the last ingestion and only re-processing those — would make the system practical for daily use. This requires tracking file hashes and updating only the affected chunks and graph edges.

6.3 Reranking with a Cross-Encoder

The current retrieval uses a bi-encoder (CodeRankEmbed) for fast embedding-based search. Adding a cross-encoder reranking step after the initial retrieval would improve precision. A cross-encoder reads the query and each candidate chunk together (not separately) and produces a more accurate relevance score, but it is too slow to run on the full index. Running it on just the top 20-30 candidates from the hybrid retriever would give the best of both worlds: fast initial retrieval plus accurate final ranking.

6.4 Graph-Aware Embeddings

Currently, the embedding model (CodeRankEmbed) has no knowledge of the graph structure. It embeds each chunk independently. Fine-tuning or adapting the embedding model to incorporate graph neighborhood information — for example, by adding the names of called functions or parent classes to the embedding text — could improve vector search quality without needing graph traversal at query time. This would effectively "bake" structural knowledge into the embeddings.

6.5 Streaming and Real-Time UI

The current web interface waits for the entire retrieval and generation pipeline to complete before showing results. Implementing streaming — where retrieved chunks appear immediately while the LLM generates the answer token-by-token — would significantly improve the user experience. FastAPI supports WebSocket connections and Server-Sent Events, both of which are compatible with streaming LLM output from Ollama.

6.6 Semantic Caching

Many developers ask similar questions about the same codebase. Implementing a semantic cache that stores (query embedding, answer) pairs and returns cached answers for semantically similar queries would reduce latency and API costs. The cache could use the same FAISS index with a similarity threshold — if a new query is within 95% cosine similarity of a cached query, return the cached answer instead of re-running the pipeline.

6.7 User Feedback Loop

Adding a thumbs-up/thumbs-down mechanism on retrieved results would create training data for improving the system over time. Positive feedback on a result could boost its score in future queries (learning-to-rank), while negative feedback could trigger re-indexing or prompt template adjustments. Over time, the system would learn which types of code chunks are most useful for different types of questions.